SDN Lab 1 Report

Lab Target

通过`Wireshark`抓包，分析各种常见协议的行为，理解网络通信的基本过程。

Report Detail

Lab Environment

- 地点：宿舍
- 网络：校园网套餐，宿舍路由器转发；
- 设备：Arch Linux

Basic Process

- 首先执行`sudo nmcli device disconnect wlan0`，手动断开连接；随后设置`Wireshark`开始抓包；
- 然后`sudo nmcli device connect wlan0`重新连接。
- 打开Chrome，输入`baidu.com`
- 几秒后停止抓包。

Protocol Analysis

DHCP

1	0.000000000	0.0.0.0	255.255.255.255	DHCP	335	DHCP Request - Transaction ID 0x2397b8a1
2	0.003068535	0.0.0.0	224.0.0.22	IGMPv3	54	Membership Report / Join group 224.0.0.251 for any sources
3	0.003147469	::	ff02::16	ICMPv6	110	Multicast Listener Report Message v2
4	0.052515465	192.168.2.1	255.255.255.255	DHCP	339	DHCP ACK - Transaction ID 0x2397b8a1
5	0.074400740	Intel_11:eb:58	Broadcast	ARP	42	Who has 192.168.2.16? (ARP Probe)
6	0.132621203	Intel_11:eb:58	Broadcast	ARP	42	Who has 192.168.2.16? (ARP Probe)
7	0.169300308	Intel_11:eb:58	Broadcast	ARP	42	Who has 192.168.2.16? (ARP Probe)
8	0.240410884	Intel_11:eb:58	Broadcast	ARP	42	ARP Announcement for 192.168.2.16
9	0.282516950	Intel_11:eb:58	Broadcast	ARP	42	Who has 192.168.2.1? Tell 192.168.2.16

标准的`DHCP`应该是包括四个过程的，分别是Discover、Offer、Request、Ack，而这里前两个过程都没有，只剩下最后两个。这是因为当前的DHCP分配的IP租约还没有过期（一般是24小时），所以在我断网重连的时候通过一个“续约”的机制快速重连，设备已知自己上次使用的IP（`192.168.2.16`），直接询问DHCP服务器是否能继续使用（图中报文1的`Transaction ID 0xabcdef...`信息就证明了这一点）。DHCP服务器发现IP未被别人使用，所以就继续分配给本机了。

ARP

arp

No.	Time	Source	Destination	Protocol	Length	Info
5	0.074400740	Intel_11:eb:58	Broadcast	ARP	42	Who has 192.168.2.16? (ARP Probe)
6	0.132621203	Intel_11:eb:58	Broadcast	ARP	42	Who has 192.168.2.16? (ARP Probe)
7	0.169300308	Intel_11:eb:58	Broadcast	ARP	42	Who has 192.168.2.16? (ARP Probe)
8	0.240410884	Intel_11:eb:58	Broadcast	ARP	42	ARP Announcement for 192.168.2.16
9	0.282516950	Intel_11:eb:58	Broadcast	ARP	42	Who has 192.168.2.1? Tell 192.168.2.16
10	0.295309941	zte_df:be:e3	Intel_11:eb:58	ARP	42	192.168.2.1 is at 54:46:17:df:be:e3
63	2.240615212	Intel_11:eb:58	Broadcast	ARP	42	ARP Announcement for 192.168.2.16
78	4.240769894	Intel_11:eb:58	Broadcast	ARP	42	ARP Announcement for 192.168.2.16
108	11.292519789	zte_df:be:e3	Intel_11:eb:58	ARP	42	Who has 192.168.2.16? Tell 192.168.2.1
109	11.292537719	Intel_11:eb:58	zte_df:be:e3	ARP	42	192.168.2.16 is at dc:46:28:11:eb:58
2069	20.590233810	ae:3c:e0:91:36:58	Broadcast	ARP	42	Who has 192.168.2.1? Tell 192.168.2.13
2946	25.914957159	ae:3c:e0:91:36:58	Broadcast	ARP	42	Who has 192.168.2.1? Tell 192.168.2.13
3035	27.041393293	62:c1:90:de:fd:22	Broadcast	ARP	42	Who has 192.168.2.1? Tell 192.168.2.8
3058	27.451271778	5e:12:db:18:80:04	Broadcast	ARP	42	Who has 192.168.2.1? Tell 192.168.2.9

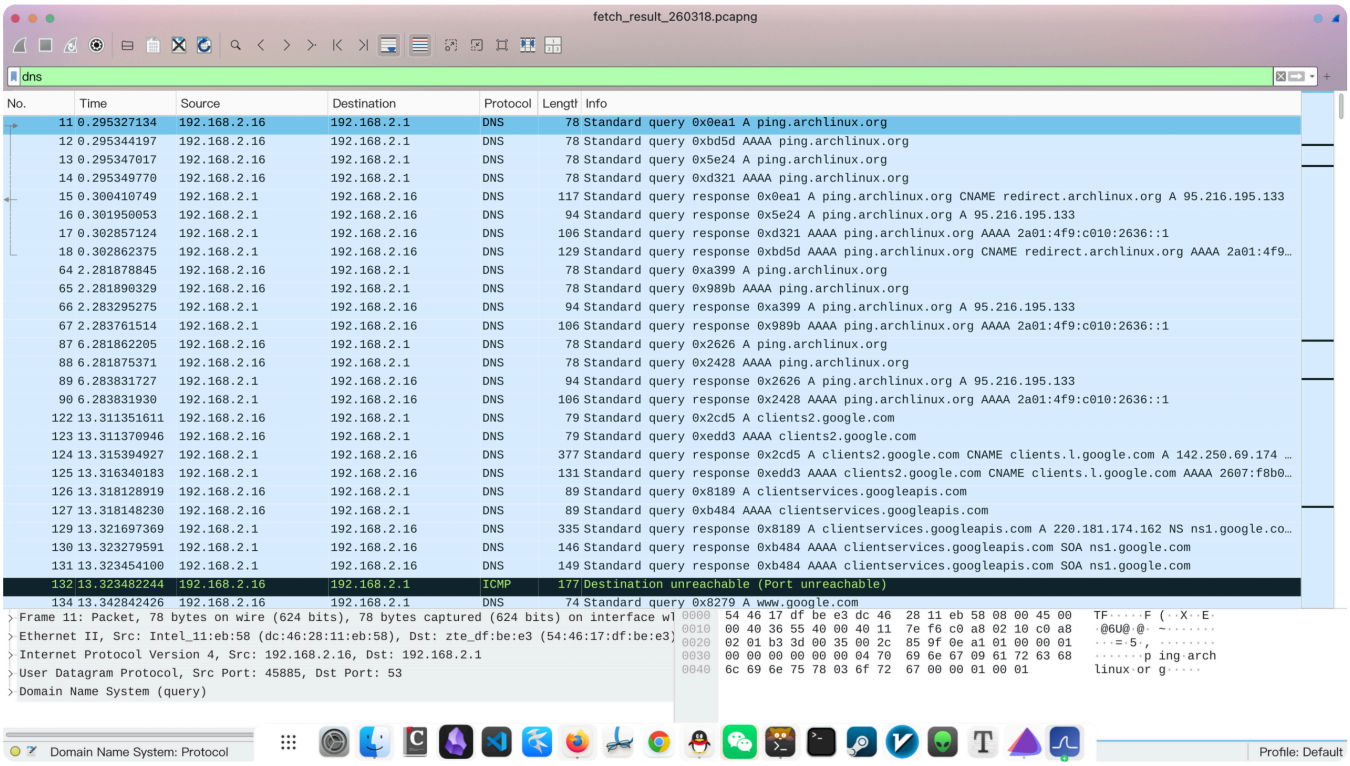
以上是所有ARP协议的包。可以分为四类：

- `5-7`：这三个包是由DHCP得到IP之后，所做的最后测试。可以看到源地址是本机的MAC地址（`Intel_11:eb:58`），这是本机在占有IP前广播发出的冲突测试，确保同一网段别的设备没有占有这个IP；
- `8`/`63`/`78`：在确定这个`.16`IP可以占领之后，本机重复广播发包，向网段中其他设备宣告这一IP被自己占用，让交换机能够更新它的MAC地址映射表，保证发往本机IP的数据流量能精准被本机接收。
- `9`/`10`：本机在确定自己IP后，立即寻找网关在哪里，于是先广播发包询问`192.168.2.1`网关地址的持有设备在哪里；网关收到这个广播查询，向本机回复自己的MAC地址。这也是比较常见的ARP的用法。
- `2069`等：别的同网段设备在广播寻找网关，本设备捕捉到了，就记录了下来。

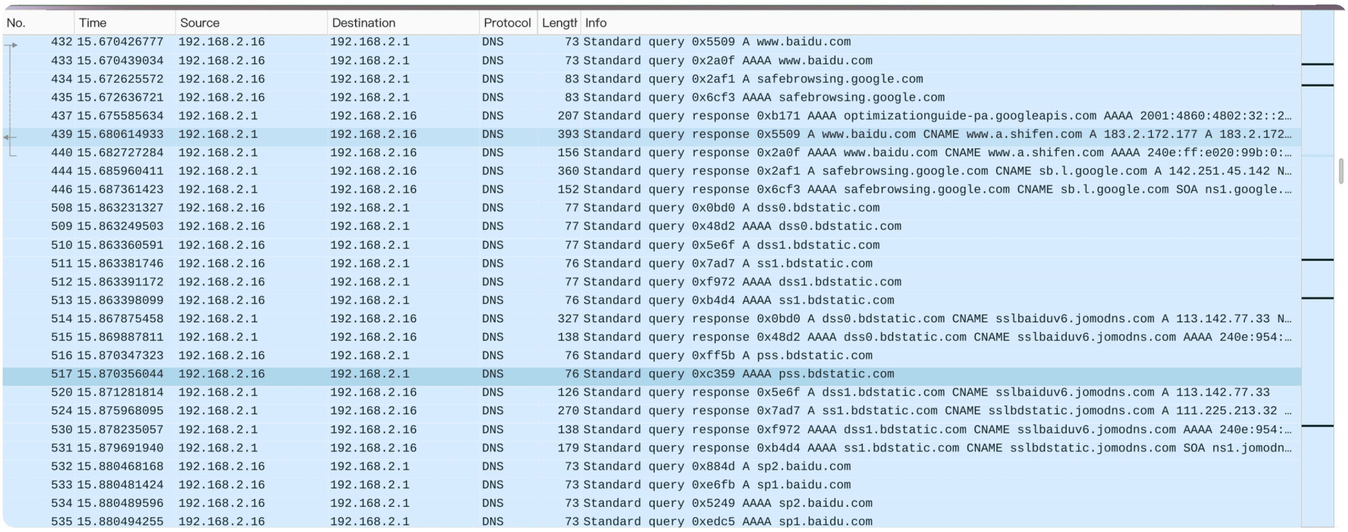
  输入命令`arp -a`，也确实能看到经过这一步后，本机成功记录了网关的MAC地址。

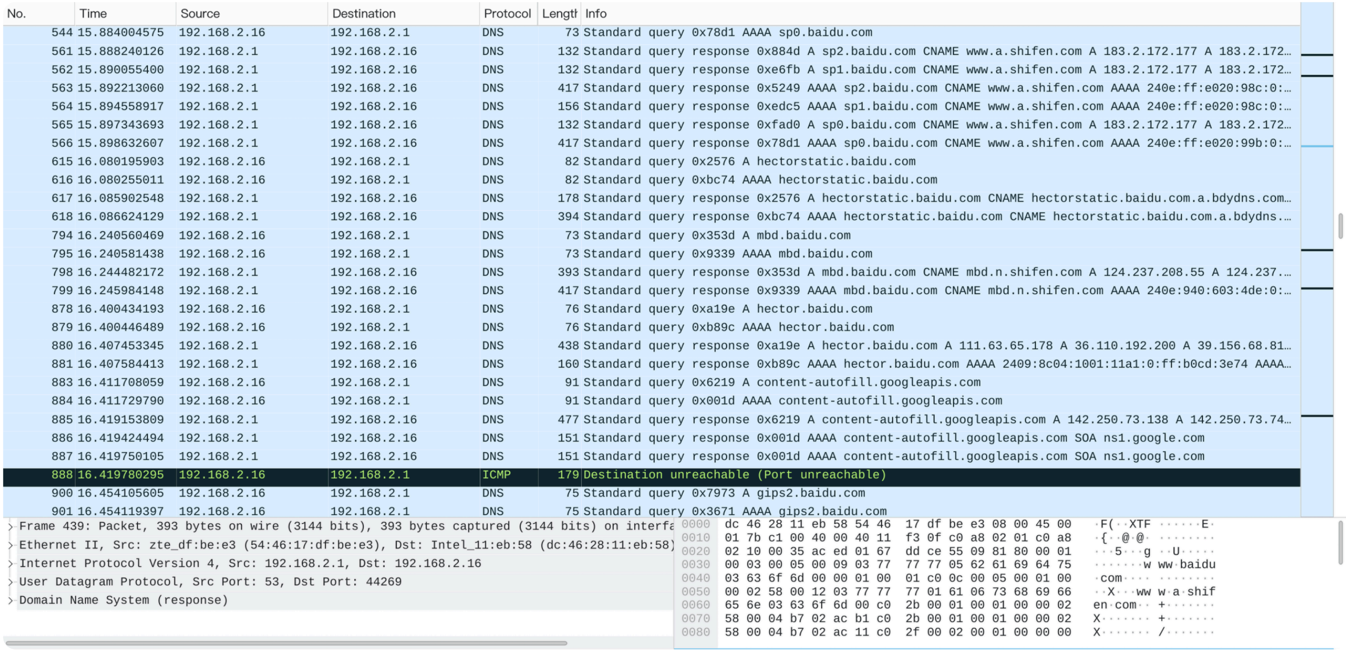
```
❯ arp -a
ziyuliu258.cn (38.38.251.209) at <incomplete> on virbr0
root-zrh-01.zerotier.com (84.17.53.155) at <incomplete> on docker0
root-tok-01.zerotier.com (79.127.159.187) at <incomplete> on docker0
_gateway (192.168.2.1) at 54:46:17:df:be:e3 [ether] on wlan0
150.217.41.34.bc.googleusercontent.com (34.41.217.150) at <incomplete> on docker0
root-zrh-01.zerotier.com (84.17.53.155) at <incomplete> on virbr0
150.217.41.34.bc.googleusercontent.com (34.41.217.150) at <incomplete> on virbr0
root-mia-01.zerotier.com (103.195.103.66) at <incomplete> on virbr0
root-tok-01.zerotier.com (79.127.159.187) at <incomplete> on virbr0
root-lax-01.zerotier.com (185.152.67.145) at <incomplete> on virbr0
ziyuliu258.cn (38.38.251.209) at <incomplete> on docker0
root-lax-01.zerotier.com (185.152.67.145) at <incomplete> on docker0
root-mia-01.zerotier.com (103.195.103.66) at <incomplete> on docker0
? (192.168.2.8) at 62:c1:90:de:fd:22 [ether] on wlan0
```

DNS



由于我的电脑有不少需要静默访问网络的服务，加上打开浏览器之后又有很多事务，所以DNS抓包记录很杂乱。下面只拣选和访问百度网有关的包分析。

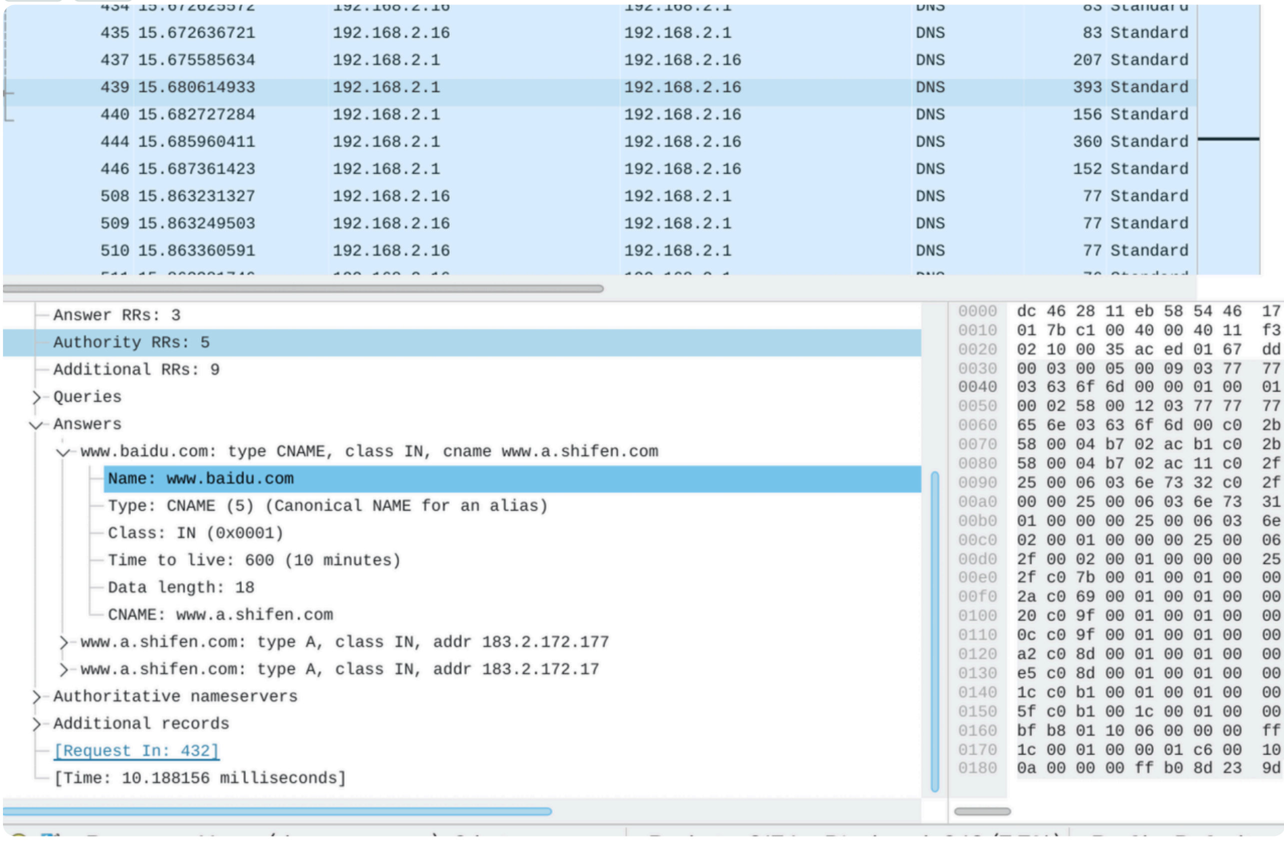


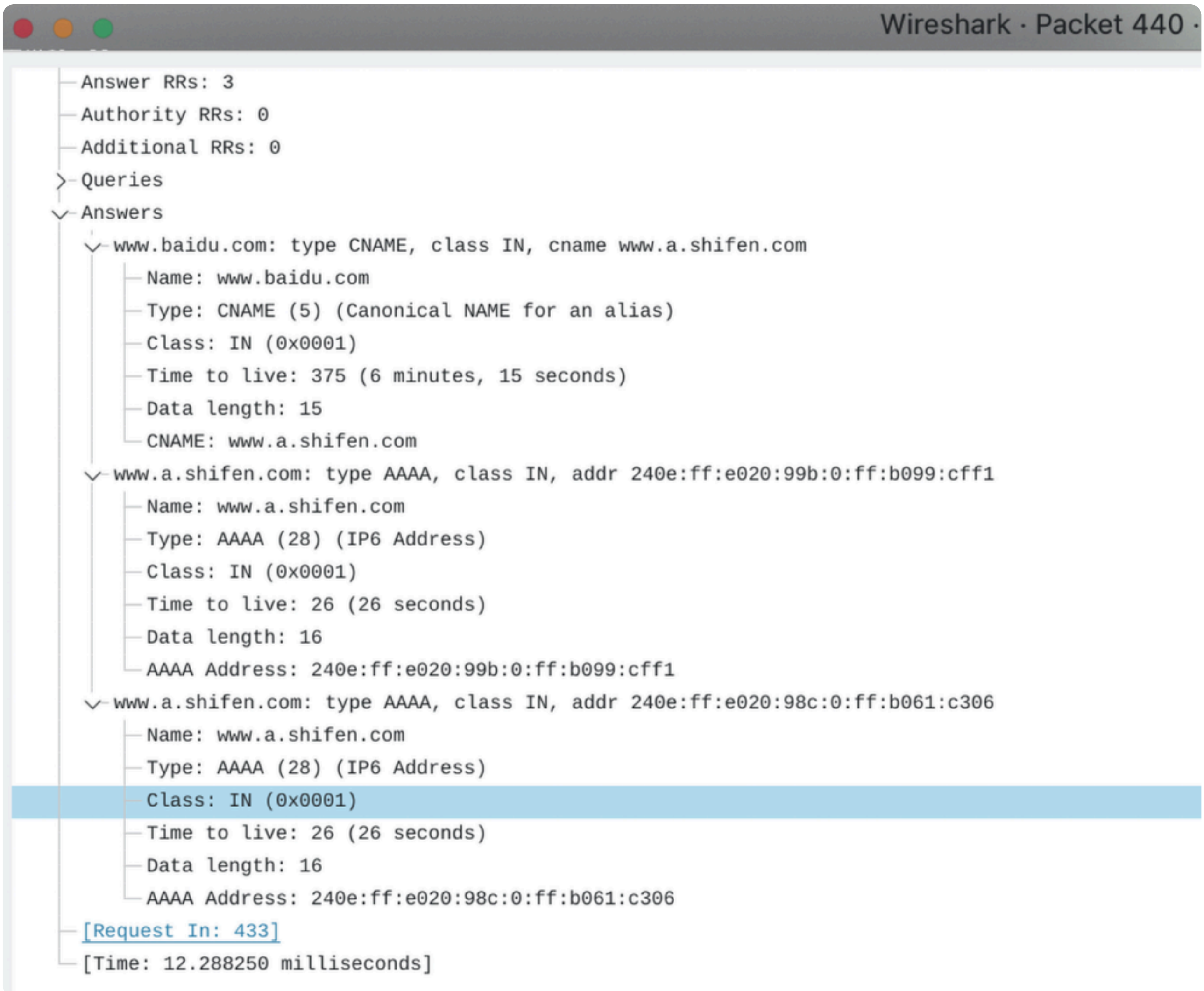


- `432` `433`

  这两个包是本机向网关发起记录查询请求，查询`www.baidu.com`的`A`和`AAAA`两种记录，对应记录百度网的`IPv4`和`IPv6`地址。两个包对应的应答包分别为`439`和`440`。

- `439` `440`

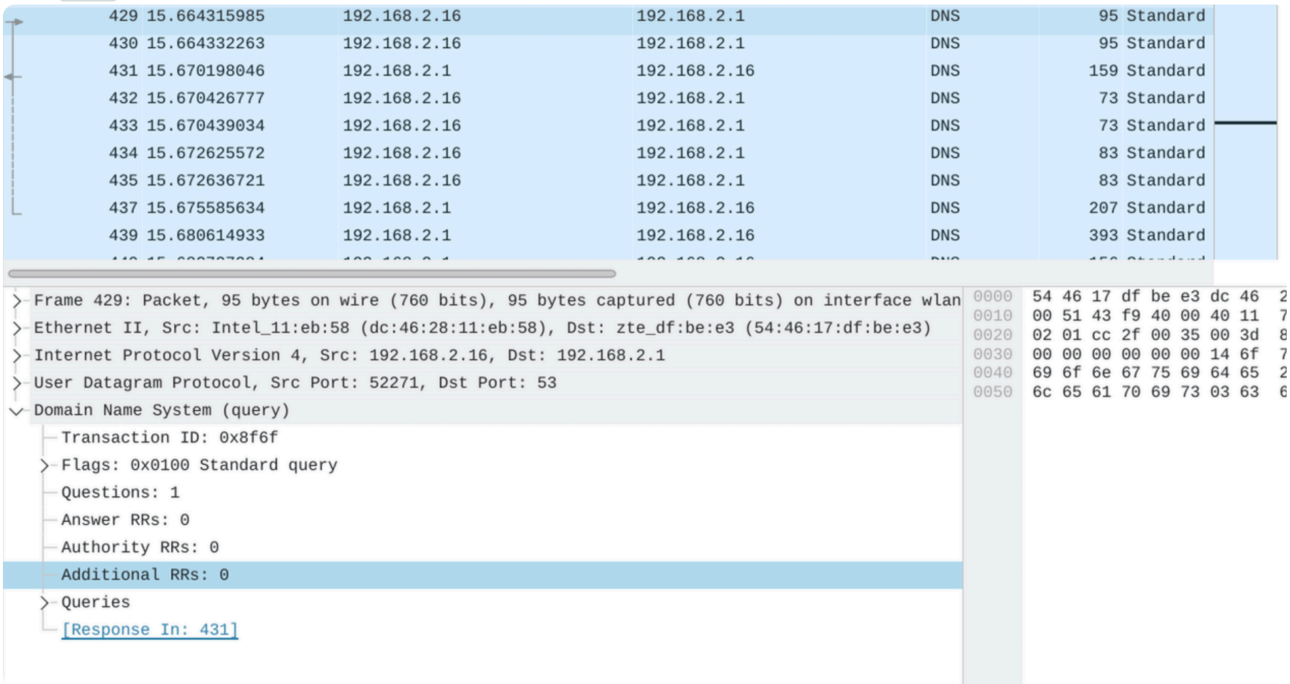




以上两图给出了两个报文的部分内容。可以看到，对于`IPv4/v6`地址的域名解析查询请求，DNS服务器返回了一个`CNAME`（Canonical Name），表明了百度网的真实域名是`www.a.shifen.com`，随后给出了从这

一域名到两条`A`/`AAAA`记录的映射。这个`CNAME`的作用就是方便将全国各地的用户引导到不同的`shifen.com`子域名上，让用户访问最近的服务器，从而实现负载均衡和流量调控。而因为路由器本身具有DNS中继/转发的功能，所以网关本身就缓存了一些DNS记录，本机在查询的时候就会直接先问网关（*因此图中的查询包的目的地址是网关IP*），网关查自己的缓存，如果有就回复给查询者，否则就代表本机去查询电信/联通等运营商的服务器，拿到结果后存到缓存返回给本机。

- `508` - `513`等
  至于后面还有一些零零碎碎和`baidu`有点关系的其他域名，这是因为百度有很多网页资源都放在别的资源服务器（如`ss1.bdstatic.com`）上，所以也要解析这些域名，用来之后渲染页面。
- 基于`UDP`



  另外，从这些DNS包的详细信息中可以看到，DNS本身属于应用层，在传输层的端口却是`53`，这说明查询请求是封装在`UDP`包里的。选择这样不可靠的协议而非`TCP`的原因是，`UDP`毕竟速度快，是无连接的，不需要双方反复连接确认；头部开销也小，能有效节省带宽；并且应用层可以用重试机制来增加查询行为的成功率，整体来说比`TCP`划算得多。（但是在特殊情况也支持`TCP`）。

UDP

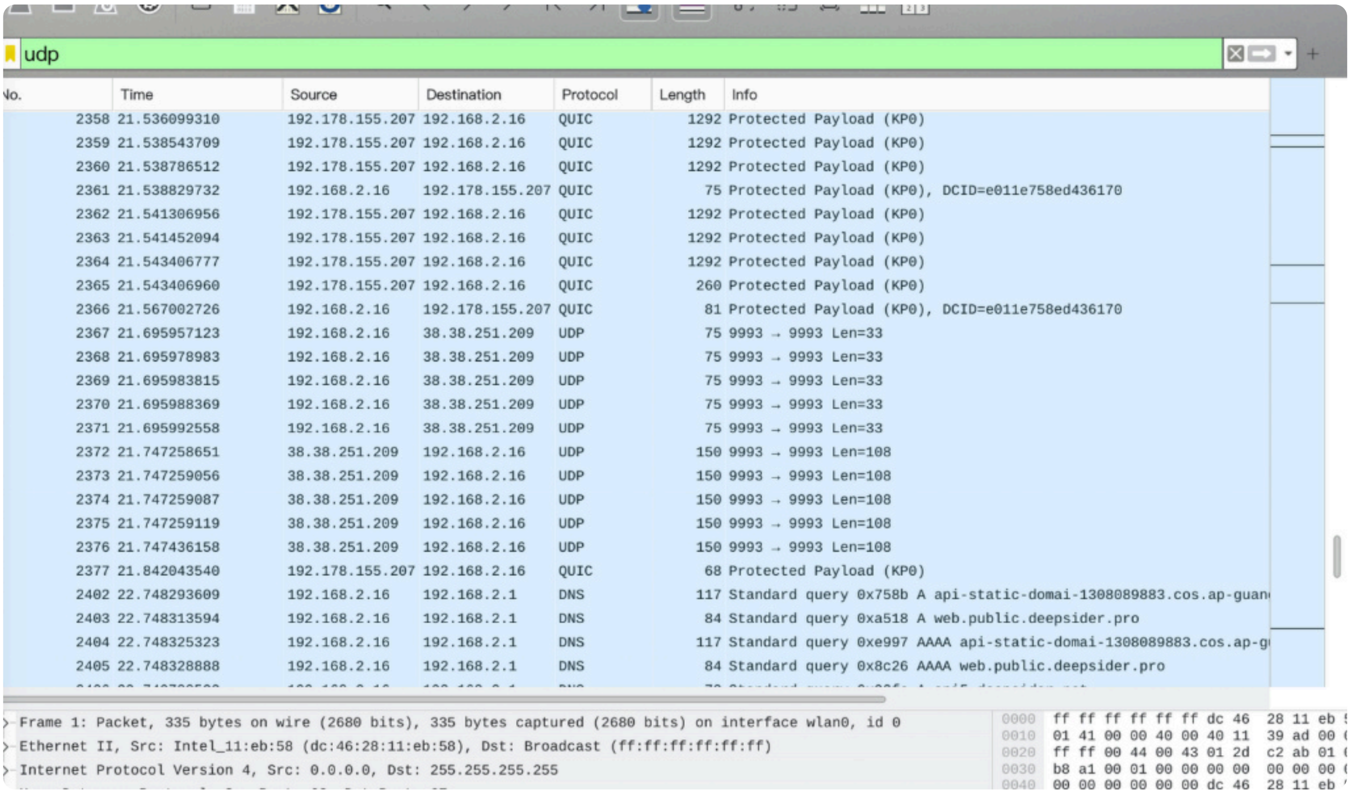
udp

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	0.0.0.0	255.255.255.255	DHCP	335	DHCP Request - Transaction ID 0x2397b8a1
4	0.052515465	192.168.2.1	255.255.255.255	DHCP	339	DHCP ACK - Transaction ID 0x2397b8a1
11	0.295327134	192.168.2.16	192.168.2.1	DNS	78	Standard query 0x0ea1 A ping.archlinux.org
12	0.295344197	192.168.2.16	192.168.2.1	DNS	78	Standard query 0xbd5d AAAA ping.archlinux.org
13	0.295347017	192.168.2.16	192.168.2.1	DNS	78	Standard query 0x5e24 A ping.archlinux.org
14	0.295349770	192.168.2.16	192.168.2.1	DNS	78	Standard query 0xd321 AAAA ping.archlinux.org
15	0.300410749	192.168.2.1	192.168.2.16	DNS	117	Standard query response 0x0ea1 A ping.archlinux.org CNAME redir
16	0.301950053	192.168.2.1	192.168.2.16	DNS	94	Standard query response 0x5e24 A ping.archlinux.org A 95.216.19
17	0.302857124	192.168.2.1	192.168.2.16	DNS	106	Standard query response 0xd321 AAAA ping.archlinux.org AAAA 2a0
18	0.302862375	192.168.2.1	192.168.2.16	DNS	129	Standard query response 0xbd5d AAAA ping.archlinux.org CNAME re
23	0.494395794	192.168.2.16	224.0.0.251	MDNS	136	Standard query 0x0000 ANY 16.2.168.192.in-addr.arpa, "QM" quest
32	0.673895692	192.168.2.16	255.255.255.255	UDP	674	47706 → 1716 Len=2112
33	0.674778090	192.168.2.16	224.0.0.251	MDNS	82	Standard query 0x0000 PTR _kdeconnect._udp.local, "QU" question
34	0.674930179	192.168.2.16	38.38.251.209	UDP	365	9993 → 9993 Len=323
35	0.674951571	192.168.2.16	34.41.217.150	UDP	365	47194 → 23013 Len=323
36	0.674987505	192.168.2.16	38.38.251.209	UDP	365	9993 → 9993 Len=323
37	0.674997566	192.168.2.16	34.41.217.150	UDP	365	47194 → 23013 Len=323
38	0.675028502	192.168.2.16	38.38.251.209	UDP	365	9993 → 9993 Len=323
39	0.675038207	192.168.2.16	34.41.217.150	UDP	365	47194 → 23013 Len=323
40	0.745474445	192.168.2.16	224.0.0.251	MDNS	136	Standard query 0x0000 ANY 16.2.168.192.in-addr.arpa, "QM" quest
45	0.996382848	192.168.2.16	224.0.0.251	MDNS	136	Standard query 0x0000 ANY 16.2.168.192.in-addr.arpa, "QM" quest
46	1.197096329	192.168.2.16	224.0.0.251	MDNS	124	Standard query response 0x0000 PTR, cache flush ziyu-dell.local
47	1.675350853	192.168.2.16	38.38.251.209	UDP	75	9993 → 9993 Len=33
48	1.675371750	192.168.2.16	38.38.251.209	UDP	75	9993 → 9993 Len=33

udp

No.	Time	Source	Destination	Protocol	Length	Info
323	14.729169619	192.168.2.16	192.168.2.1	DNS	71	Standard query 0x56f8 AAAA s.deepl.com
324	14.732790441	192.168.2.1	192.168.2.16	DNS	106	Standard query response 0x1ffa A s.deepl.com CNAME s-lb.deepl.c
325	14.733238576	192.168.2.1	192.168.2.16	DNS	170	Standard query response 0x56f8 AAAA s.deepl.com CNAME s-lb.deep
348	14.974975594	104.18.36.122	192.168.2.16	QUIC	70	Protected Payload (KP0)
351	15.000857829	192.168.2.16	104.18.36.122	QUIC	86	Protected Payload (KP0), DCID=015c04ac00c21224bb5dbbaca9c22327a
402	15.474200191	192.168.2.16	199.16.158.182	QUIC	1292	Initial, DCID=2f0db1948e187ecb, PKN: 8, CRYPTO, PING, CRYPTO, P
429	15.664315985	192.168.2.16	192.168.2.1	DNS	95	Standard query 0x8f6f A optimizationguide-pa.googleapis.com
430	15.664332263	192.168.2.16	192.168.2.1	DNS	95	Standard query 0xb171 AAAA optimizationguide-pa.googleapis.com
431	15.670198046	192.168.2.1	192.168.2.16	DNS	159	Standard query response 0x8f6f A optimizationguide-pa.googleapi
432	15.670426777	192.168.2.16	192.168.2.1	DNS	73	Standard query 0x5509 A www.baidu.com
433	15.670439034	192.168.2.16	192.168.2.1	DNS	73	Standard query 0x2a0f AAAA www.baidu.com
434	15.672625572	192.168.2.16	192.168.2.1	DNS	83	Standard query 0x2af1 A safebrowsing.google.com
435	15.672636721	192.168.2.16	192.168.2.1	DNS	83	Standard query 0x6cf3 AAAA safebrowsing.google.com
436	15.673632062	192.168.2.16	74.125.142.84	QUIC	1292	Initial, DCID=95f0943a88886d22, PKN: 8, CRYPTO, PING, CRYPTO, P
437	15.675585634	192.168.2.1	192.168.2.16	DNS	207	Standard query response 0xb171 AAAA optimizationguide-pa.google
439	15.680614933	192.168.2.1	192.168.2.16	DNS	393	Standard query response 0x5509 A www.baidu.com CNAME www.a.shif
440	15.682727284	192.168.2.1	192.168.2.16	DNS	156	Standard query response 0x2a0f AAAA www.baidu.com CNAME www.a.s
444	15.685960411	192.168.2.1	192.168.2.16	DNS	360	Standard query response 0x2af1 A safebrowsing.google.com CNAME
445	15.686546037	192.168.2.16	142.250.69.170	QUIC	1292	Initial, DCID=845d02871a29be7a, PKN: 8, PING, PADDING, CRYPTO,
446	15.687361423	192.168.2.1	192.168.2.16	DNS	152	Standard query response 0x6cf3 AAAA safebrowsing.google.com CNA
508	15.863231327	192.168.2.16	192.168.2.1	DNS	77	Standard query 0x0bd0 A dss0.bdstatic.com
509	15.863249503	192.168.2.16	192.168.2.1	DNS	77	Standard query 0x48d2 AAAA dss0.bdstatic.com
510	15.863360591	192.168.2.16	192.168.2.1	DNS	77	Standard query 0x5e6f A dss1.bdstatic.com
511	15.863381746	192.168.2.16	192.168.2.1	DNS	76	Standard query 0x7ad7 A ss1.bdstatic.com

> Frame 1: Packet, 335 bytes on wire (2680 bits), 335 bytes captured (2680 bits) on interface wlan0, id 0 | 0000 ff ff ff ff ff ff dc 46 28 11 eb

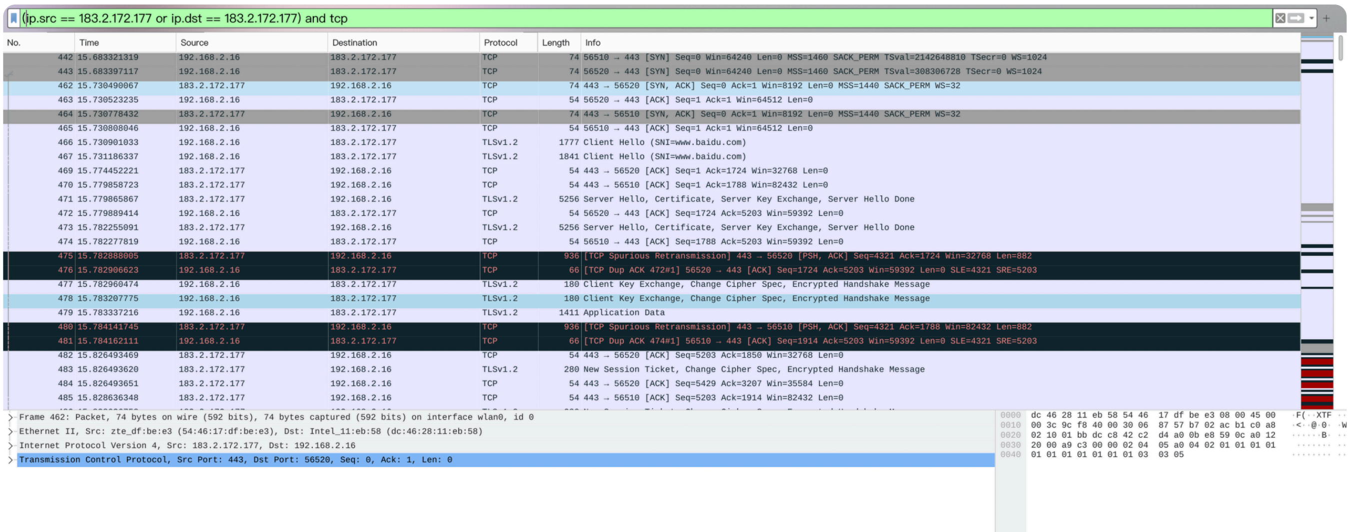


UDP筛选出来的报文列表是最乱的。很多协议似乎都以此为载体。

1. `QUIC`：有大量 QUIC 协议报文，似乎看不出和什么服务有关，查询网上的资料，给出的说法是一种新型的用 `UDP` 封装的带可靠传输和加密功能的协议。
2. `MDNS`：MDNS 协议也通过 `UDP` 封装，为了发现局域网中其他的设备。
3. 虚拟组网：捕获到很多和 9993 端口 有关的流量，结合上图 `arp -a` 的结果，推测可能是本机的 `Zerotier` 在通过 UDP 穿透 NAT、和VLAN上其他的设备尝试建立连接。

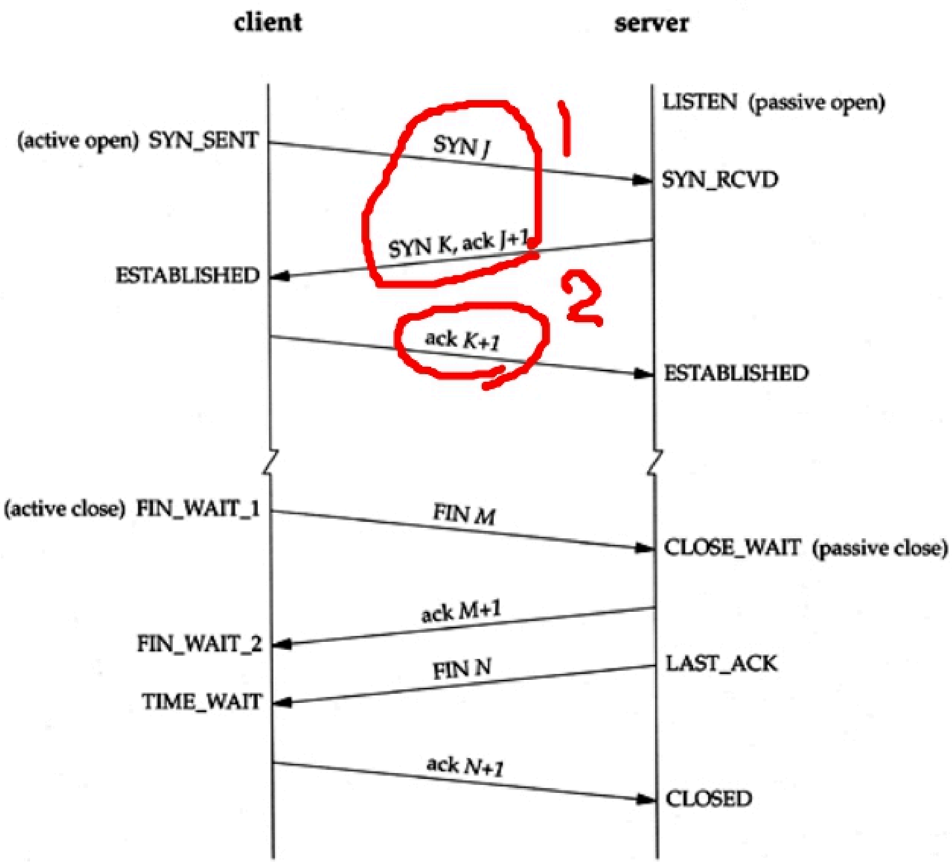
TCP

Build A Connection



这个图中可以见到很清晰的“三次握手”建立连接的过程，对应三个包分别是 `443` `462` 和 `463` 。对照下面的图就

可以看出来了。



Keep It Alive, then FIN

No.	Time	Source	Destination	Protocol	Length	Info
606	18.407046219	183.2.172.17	192.168.2.16	TLSv1.2	603	Application Data
607	18.407067448	192.168.2.16	183.2.172.17	TCP	54	37876 → 443 [ACK] Seq=3605 Ack=5978 Win=59392 Len=0
649	18.523133590	192.168.2.16	183.2.172.17	TLSv1.2	1152	Application Data
660	18.556721381	183.2.172.17	192.168.2.16	TCP	60	443 → 37840 [ACK] Seq=162044 Ack=6070 Win=93952 Len=0
661	18.559739801	183.2.172.17	192.168.2.16	TLSv1.2	713	Application Data
662	18.559773173	192.168.2.16	183.2.172.17	TCP	54	37840 → 443 [ACK] Seq=6070 Ack=162703 Win=230400 Len=0
2794	33.287393804	183.2.172.17	192.168.2.16	TCP	54	[TCP Keep-Alive] 443 → 37846 [ACK] Seq=5428 Ack=1850 Win=82304 Len=0
2795	33.287415129	192.168.2.16	183.2.172.17	TCP	54	[TCP Keep-Alive ACK] 37846 → 443 [ACK] Seq=1850 Ack=5429 Win=59392 Len=0
2796	33.438531107	183.2.172.17	192.168.2.16	TCP	54	[TCP Keep-Alive] 443 → 37862 [ACK] Seq=5977 Ack=3635 Win=85888 Len=0
2797	33.438548197	192.168.2.16	183.2.172.17	TCP	54	[TCP Keep-Alive ACK] 37862 → 443 [ACK] Seq=3635 Ack=5978 Win=59392 Len=0
2798	33.449208212	183.2.172.17	192.168.2.16	TCP	54	[TCP Keep-Alive] 443 → 37876 [ACK] Seq=5977 Ack=3605 Win=38528 Len=0
2799	33.449230666	192.168.2.16	183.2.172.17	TCP	54	[TCP Keep-Alive ACK] 37876 → 443 [ACK] Seq=3605 Ack=5978 Win=59392 Len=0
2801	33.598858401	183.2.172.17	192.168.2.16	TCP	60	[TCP Keep-Alive] 443 → 37840 [ACK] Seq=162702 Ack=6070 Win=93952 Len=0
2802	33.598877301	192.168.2.16	183.2.172.17	TCP	54	[TCP Keep-Alive ACK] 37840 → 443 [ACK] Seq=6070 Ack=162703 Win=230400 Len=0
2805	33.722680287	183.2.172.17	192.168.2.16	TCP	54	[TCP Keep-Alive] 443 → 37882 [ACK] Seq=5977 Ack=5021 Win=39040 Len=0
2806	33.722702107	192.168.2.16	183.2.172.17	TCP	54	[TCP Keep-Alive ACK] 37882 → 443 [ACK] Seq=5021 Ack=5978 Win=59392 Len=0

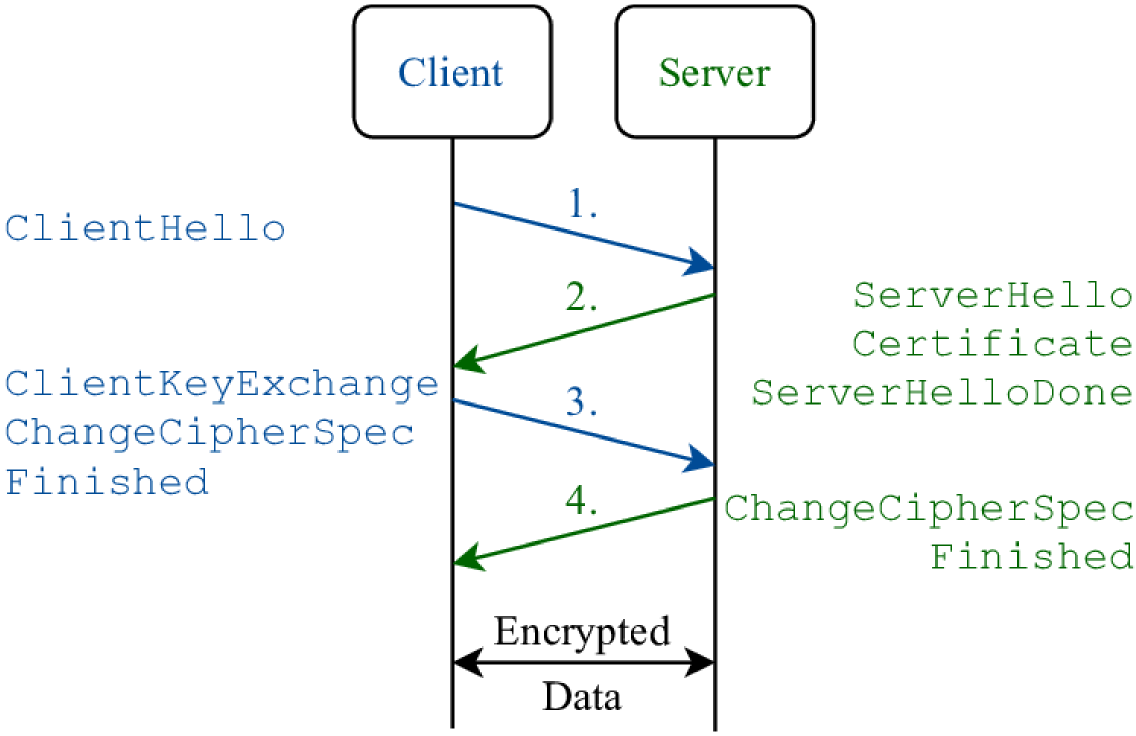
No.	Time	Source	Destination	Protocol	Length	Info
727	9.742977144	192.168.2.16	183.2.172.17	TCP	54	44812 → 443 [RST] Seq=3602 Win=0 Len=0
728	9.743063326	183.2.172.17	192.168.2.16	TLSv1.2	85	Encrypted Alert
729	9.743063461	183.2.172.17	192.168.2.16	TCP	54	443 → 44800 [FIN, ACK] Seq=5460 Ack=1851 Win=82304 Len=0
730	9.743091733	192.168.2.16	183.2.172.17	TCP	54	44800 → 443 [RST] Seq=1851 Win=0 Len=0
731	9.743102860	192.168.2.16	183.2.172.17	TCP	54	44800 → 443 [RST] Seq=1851 Win=0 Len=0
909	18.429419780	192.168.2.16	183.2.172.17	TCP	74	59536 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=3309638644 TSecr=0 WS=1024
910	18.464823844	183.2.172.17	192.168.2.16	TCP	74	443 → 59536 [SYN, ACK] Seq=0 Ack=1 Win=8192 Len=0 MSS=1440 SACK_PERM WS=32
911	18.464868663	192.168.2.16	183.2.172.17	TCP	54	59536 → 443 [ACK] Seq=1 Ack=1 Win=64512 Len=0
912	18.465189803	192.168.2.16	183.2.172.17	TLSv1.2	1969	Client Hello (SNI=www.baidu.com)
913	18.498281491	183.2.172.17	192.168.2.16	TCP	54	443 → 59536 [ACK] Seq=1 Ack=1916 Win=82688 Len=0
914	18.498281661	183.2.172.17	192.168.2.16	TLSv1.2	212	Server Hello, Change Cipher Spec, Encrypted Handshake Message
915	18.498314608	192.168.2.16	183.2.172.17	TCP	54	59536 → 443 [ACK] Seq=1916 Ack=159 Win=64512 Len=0
916	18.501027502	192.168.2.16	183.2.172.17	TLSv1.2	105	Change Cipher Spec, Encrypted Handshake Message
917	18.501174167	192.168.2.16	183.2.172.17	TLSv1.2	1792	Application Data
918	18.533505502	183.2.172.17	192.168.2.16	TCP	54	443 → 59536 [ACK] Seq=159 Ack=1967 Win=82688 Len=0
919	18.533505988	183.2.172.17	192.168.2.16	TCP	54	443 → 59536 [ACK] Seq=159 Ack=3705 Win=86272 Len=0
920	18.542210026	183.2.172.17	192.168.2.16	TLSv1.2	403	Application Data
921	18.583215384	192.168.2.16	183.2.172.17	TCP	54	59536 → 443 [ACK] Seq=3705 Ack=508 Win=64512 Len=0
1295	33.850325488	183.2.172.17	192.168.2.16	TCP	54	[TCP Keep-Alive] 443 → 59536 [ACK] Seq=507 Ack=3705 Win=86272 Len=0
1296	33.850359012	192.168.2.16	183.2.172.17	TCP	54	[TCP Keep-Alive ACK] 59536 → 443 [ACK] Seq=3705 Ack=508 Win=64512 Len=0
1425	49.210853736	183.2.172.17	192.168.2.16	TCP	54	[TCP Keep-Alive] 443 → 59536 [ACK] Seq=507 Ack=3705 Win=86272 Len=0
1426	49.210889160	192.168.2.16	183.2.172.17	TCP	54	[TCP Keep-Alive ACK] 59536 → 443 [ACK] Seq=3705 Ack=508 Win=64512 Len=0
1492	64.563212772	192.168.2.16	183.2.172.17	TCP	54	[TCP Keep-Alive] 59536 → 443 [ACK] Seq=3704 Ack=508 Win=64512 Len=0
1493	64.570145619	183.2.172.17	192.168.2.16	TCP	54	[TCP Keep-Alive] 443 → 59536 [ACK] Seq=507 Ack=3705 Win=86272 Len=0
1494	64.570170070	192.168.2.16	183.2.172.17	TCP	54	[TCP Keep-Alive ACK] 59536 → 443 [ACK] Seq=3705 Ack=508 Win=64512 Len=0
1496	64.597120497	183.2.172.17	192.168.2.16	TCP	54	[TCP Dup ACK 919#1] 443 → 59536 [ACK] Seq=508 Ack=3705 Win=86272 Len=0
1625	79.930956728	183.2.172.17	192.168.2.16	TCP	54	[TCP Keep-Alive] 443 → 59536 [ACK] Seq=507 Ack=3705 Win=86272 Len=0
1626	79.930992479	192.168.2.16	183.2.172.17	TCP	54	[TCP Keep-Alive ACK] 59536 → 443 [ACK] Seq=3705 Ack=508 Win=64512 Len=0
1730	95.290304401	183.2.172.17	192.168.2.16	TCP	54	[TCP Keep-Alive] 443 → 59536 [ACK] Seq=507 Ack=3705 Win=86272 Len=0
1731	95.290340943	192.168.2.16	183.2.172.17	TCP	54	[TCP Keep-Alive ACK] 59536 → 443 [ACK] Seq=3705 Ack=508 Win=64512 Len=0
1782	103.542214965	183.2.172.17	192.168.2.16	TLSv1.2	85	Encrypted Alert
1783	103.542256212	192.168.2.16	183.2.172.17	TCP	54	59536 → 443 [ACK] Seq=3705 Ack=539 Win=64512 Len=0
1784	103.542463041	183.2.172.17	192.168.2.16	TCP	54	443 → 59536 [FIN, ACK] Seq=539 Ack=3705 Win=86272 Len=0
1785	103.571924586	183.2.172.17	192.168.2.16	TCP	54	[TCP Retransmission] 443 → 59536 [FIN, ACK] Seq=539 Ack=3705 Win=86272 Len=0
1786	103.571950268	192.168.2.16	183.2.172.17	TCP	66	59536 → 443 [ACK] Seq=3705 Ack=540 Win=64512 Len=0 SLE=539 SRE=540
1803	106.607996594	183.2.172.17	192.168.2.16	TCP	54	443 → 59536 [RST] Seq=540 Win=0 Len=0

由于TCP Keep-Alive机制的存在，一个优雅的“四次挥手”过程不太容易被抓到。上图是我补测了两次后勉强得到的一个结果。首先经历了一长串的`Keep-Alive`包未有回应，百度服务器在`1784`报文处发起了`FIN`的断联请

求，随后`1782`处`TLS`也提示要断开应用层安全连接；`1786`处本机回复了`ACK`，代表接收到断联请求，随后就结束了连接。

TLS

在上图中，我们还看到了很多关于`TLS1.2`协议的内容，这是因为在输入`baidu.com`的时候，浏览器默认用`https`而不是`http`拼接成完整`URL`，所以就会引入`HTTPS`（*HTTP Over TLS*）中带有的`TLS`（传输层安全协议）握手环节。



如上图所示，`TLS1.2`的握手大致分为以上四个阶段，对应上图中的`466` `471` `477` `478`。在`478`完成之后，加密通道就会被建好，随后的`479`就是数据传输阶段，右侧信息显示`Application Data`，此时只能看到一堆乱码，真实的内容被加密运输。所以，在抓包结果里自然也就看不到什么`GET`/`POST`/`HTTP/1.1`之类的包头了。

STP

本次抓包并没有抓到STP报文。通过查询资料，推测可能是因为我们宿舍使用路由器转发了一个人的校园网套餐，这种情况下宿舍所有人的设备流量都从路由器一个点发出来，路由器处理NAT的时候不会转发STP协议，因此也就接收不到。

Process Summary

回顾以上的结果，可以粗略地如下阐述设备从入网到成功与百度网通信的流程：

1. 首先设备入网，通过`DHCP`从路由器获取了自己的内网IP地址；
2. 然后设备通过`ARP`协议确认IP和网段内其他设备不冲突，并找到了网关的MAC地址；
3. 随后，在访问`baidu.com`的时候，主机通过`DNS`协议向网关发起域名解析查询，网关返回自身缓存信息或者代主机向外查询；最后主机得到域名对应的真实的IP地址；
4. 随后通过`TCP`协议和目标建立可靠的链接，并通过`TLS`握手建立加密安全隧道，开始数据传输。

总之，这次实验对我来说还是收获颇丰，首先我通过分析报文，对`CDN`的域名重定向操作有更深理解，明白了如何利用`CNAME`来实现流量调控和负载均衡，这也让我在自己购置个人域名时设置权威解析记录的时候能够明

白自己在做什么；其次我从实际抓包流程中加深了对`TCP Keep-Alive`机制的了解，也打破了我对于TCP“四次挥手”断联的刻板印象；另外我也看到了`UDP`用途的多样性，看到它在各种现代协议中的重要作用。